

Neural Virtual Analog Modeling in Csound via Real-Time Recurrent Inference

Ken Kobayashi

Independent
kenkoba02@icloud.com

Abstract. Csound’s native virtual analog opcodes rely on fixed algorithmic structures that cannot learn transfer functions from empirical data or capture physical analog hardware. We present a real-time neural inference pipeline integrating Csound’s C++ Plugin Opcode Framework (CPOF) with the RTNeural engine, executing stateful recurrent networks directly within Csound orchestra syntax. We evaluate the pipeline on two targets: a parameterized 4-pole Moog ladder filter trained on oversampled numerical simulations, and a fixed-setting analog distortion pedal trained on physical loopback recordings. For the filter, shortening the loss-masked training warmup to 256 samples forces rapid convergence from a cold hidden state, enabling click-free note-on with only a brief output ramp; reducing the warmup further to 128 samples reduces static ESR at the 100 Hz cutoff by 7.6 dB. A multi-rate LFO training regime addresses dynamic tracking degradation: a static-trained 128-unit baseline degrades by 17.3 dB under a 5 Hz cutoff sweep, while multi-rate LFO training compresses rate-dependent variance to 3.6 dB across 1–20 Hz on held-out audio. The same recurrent architecture, without the cutoff input, achieves -29.3 dB held-out ESR and -37.9 dB aliasing-to-signal ratio at 8 kHz on the hardware distortion pedal when trained via pre-emphasized ESR on loopback recordings. Deployed 128-unit Gated Recurrent Unit (GRU) opcodes consume under 17% CPU per voice, sustaining 6-voice polyphony at 48 kHz.

Keywords: Virtual Analog Modeling, Recurrent Neural Networks, Black-Box Modeling, Csound Opcodes

1 Introduction

Virtual Analog (VA) modeling in Csound has historically relied on algorithmic DSP: native opcodes such as `moogladder` implement discretized differential equations with first-order Euler integration and $2\times$ oversampling [1]. Such opcodes cannot adapt to empirical circuit measurements or emulate non-parametric physical hardware from audio recordings. We introduce a real-time neural inference pipeline for Csound built on the C++ Plugin Opcode Framework (CPOF) and RTNeural [2] — a lightweight, header-only C++ library for running pre-trained neural network inference with low per-sample overhead, designed to be embedded directly in real-time audio code — and demonstrate its generality across two targets: a parameterized 4-pole Moog ladder filter trained on oversampled numerical simulations, and a fixed-setting analog distortion pedal trained on loopback recordings. Two findings emerge from these case studies: for the GRU-based architecture and Moog filter target evaluated here, dynamic parameter-tracking degradation was traceable to the training-distribution rather than to the one architectural alternative tested (FiLM conditioning), and cold-start convergence can be trained via short warmup windows rather than concealed with runtime fades.

2 System Architecture

Real-time audio-rate inference requires minimizing per-sample overhead. The neural graph is constructed using RTNeural’s compile-time template API (`ModelT`) [2]: the network’s layer sizes, weights, and activations are all fixed into the compiled program itself rather than read and interpreted while the plugin is running, letting the compiler optimize the resulting per-sample math (loop unrolling and SIMD vectorization). Because `ModelT` requires static hidden dimensions, a single compiled binary cannot resize its recurrent state; we template the opcode class across explicit hidden sizes ($H \in \{32, 64, 128, 256\}$), registering four distinct opcodes (`moognn32`, `moognn64`, `moognn128`, `moognn256`) so users select their cost–accuracy tradeoff directly in the orchestra. Two additional

host-level details are worth naming: (i) file-system JSON weight loading incurs ≈ 8 ms of parsing jitter on first note-on, which we hide by exposing a dedicated `moognn_preload` opcode, invoked at score time $t = 0$, that parses the weight file once and holds the result in an in-memory cache; and (ii) Csound recycles instrument memory across sequential note events without re-invoking C++ constructors, so all GRU state registers and fade counters must be explicitly zeroed in the CPOF `init()` pass to avoid stale-state clicks on notes 3+.

`moognn_preload` populates this cache once at score time $t = 0$; `moognn128` then reads the same pre-parsed model from it on every note-on, skipping the JSON parse entirely, as shown in the following orchestra fragment:

```
; score time 0: parse weights into cache
iignore moognn_preload "path/to/moog/weights.json"

; per-note: log-scaled CC cutoff, audio output
kcutoff = 100 * pow(200, kcc) ; 100 Hz to 20 kHz
aout    moognn128 ain, "path/to/moog/weights.json", kcutoff
```

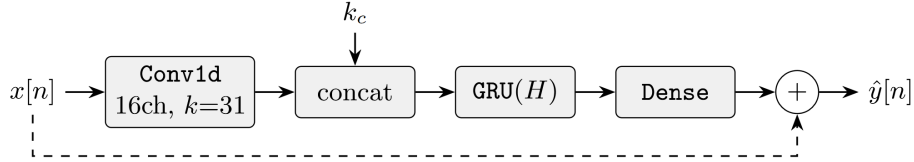


Fig. 1. Model architecture (`moognn`). A causal convolution extracts local features; the log-normalized cutoff $k_c \in [0, 1]$ is concatenated before the GRU. The dashed residual skip bypasses the recurrent core. `distnn` uses the same graph without the k_c input (16-channel GRU input).

3 Parameterized Moog Ladder Emulation

Because an exact numerical reference (the RK4 simulation) is available, the parameterized filter provides a controlled environment for investigating training strategies for real-time parametric recurrent models; the findings in Sections 3.1–3.2 apply equally to cases where no analytical model exists.

The model processes audio sample-by-sample: a 16-channel causal 1D convolution (`CausalConv1d`, kernel 31) extracts local features, a log-normalized cutoff parameter ($0 \leq k_c \leq 1$, mapping 100 Hz to 20 kHz) is concatenated to the 16 feature channels, and the resulting 17-channel vector feeds a single-layer GRU, followed by a linear projection and a dry/wet residual skip. The GRU is a recurrent layer that maintains a running internal state across samples, so its output at each sample depends on recent input history rather than the current sample alone. Training targets are generated offline by filtering benchmark audio through an $8\times$ oversampled Runge–Kutta integration of the Huovilainen [1] Moog ladder model, using the `RKSimulationMoog` implementation from [12]; 260 s of source audio filtered at 12 log-spaced static cutoffs from 100 Hz to 20 kHz yields ≈ 52 minutes of paired training data, with a 40 s bench source held out for validation. Optimization uses truncated backpropagation through time on 8192-sample windows with a 256-sample loss-masked prefix, Adam at an initial learning rate of 10^{-3} under a `ReduceLROnPlateau` schedule (factor 0.5, patience 20 epochs, minimum 10^{-6}), and global gradient clipping at max-norm 1.0. The model is optimized on Error-to-Signal Ratio (ESR) with a high-pass pre-emphasis filter $H(z) = 1 - 0.95z^{-1}$ [4], forcing the optimizer to prioritize filter rolloff over low-frequency dominance:

$$\mathcal{L}_{\text{ESR}} = \frac{1}{N} \sum_{i=1}^N \frac{\sum_n (\hat{y}_i[n] - y_i[n])^2}{\max(\sum_n y_i[n]^2, 10^{-4})} \quad (1)$$

where $\hat{y}_i$ and y_i are the pre-emphasized prediction and target for window i , n ranges over the $W = 8192$ scored samples (excluding the 256-sample warmup prefix), and the 10^{-4} energy floor prevents high-cutoff signals from dominating the denominator.

3.1 The Warmup Period-Visibility Floor

Evaluating a GRU from an uninitialized state ($\mathbf{h}_0 = \mathbf{0}$) produces audible transient clicks on note-on, a manifestation of the exposure bias between training and inference state distributions identified by Peussa et al. [3]. Rather than their remedy of learning an affine mapping from ground-truth circuit states into the GRU hidden state (which requires observable internal states of the target device), or concealing convergence with a long fade-in at runtime, we shorten the loss-masked warmup prefix within each training segment so the network is forced to learn rapid convergence from zero, then apply only a brief output ramp during live rendering. A 256-sample warmup (≈ 5.3 ms at 48 kHz) is the empirical floor. At 48 kHz, a 100 Hz signal has a period of 480 samples, so 256 samples exposes 0.53 periods to the recurrent core before loss scoring begins; further reducing the warmup to 128 samples reduces static ESR at the 100 Hz cutoff by 7.6 dB.

3.2 Dynamic Tracking and Multi-Rate LFO Training

Models trained exclusively on static cutoff frequencies exhibit severe tracking degradation under fast parameter sweeps (Fig. 2). A 128-unit baseline achieving -46.7 dB static ESR at 1 kHz degrades by 17.3 dB under a 5 Hz sinusoidal LFO cutoff sweep. Architectural interventions, including Feature-wise Linear Modulation (FiLM) placed both pre- and post-GRU [5], did not help; conditioning placement cannot circumvent the recurrent core’s inherent need to integrate parameter changes through its hidden state, and the true bottleneck is distributional mismatch. Extending the training set with five time-varying LFO sweeps from 1 Hz to 20 Hz [11] (adding ≈ 22 minutes of dynamic training pairs) eliminates dynamic lag, compressing total error variance across the same rate range to 3.6 dB while also modestly improving static accuracy, most notably at low frequencies where the static-only baseline was weakest (Table 1 reports a 3.4 dB improvement in the mean across all evaluated frequencies).

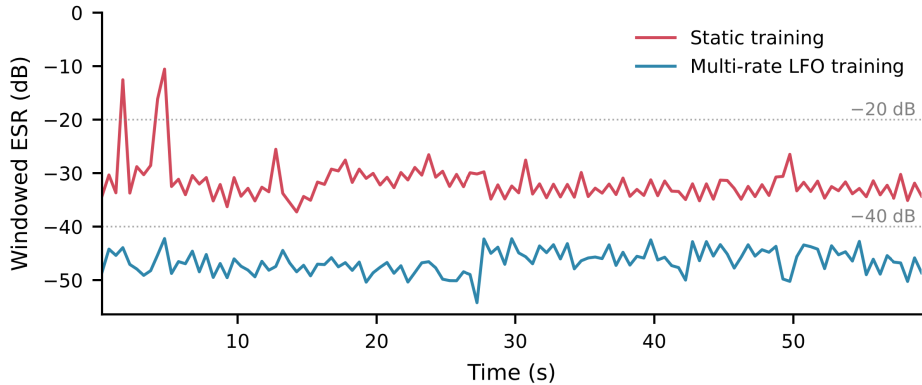


Fig. 2. Windowed ESR (0.5 s) under a 5 Hz LFO cutoff sweep on held-out audio. The 128-unit model trained on static cutoffs alone sits ≈ 15 dB above the same architecture trained with multi-rate LFO sweeps added.

4 Non-Parametric Hardware Distortion

To validate that the framework generalizes beyond simulated targets, we model a hardware breadboard distortion effect (`distnn`): a TL072 opamp with diodes on the feedback path (two 1N34A germanium diodes in series on one polarity, one BAT46 Schottky on the other) operated at high gain, producing aggressive asymmetric soft-clipping. Black-box neural network modeling of distortion circuits from audio recordings was established as a viable real-time approach by Damsk gg et al. [4] and Wright et al. [6]; our work deploys the same class of model as a Csound opcode. `distnn` shares the `moognn` graph (Fig. 1) with the k_c input removed. Dry and wet audio are captured simultaneously through a two-channel loopback at 0 dB and -10 dB input levels; because both channels share the interface clock, ADC/DAC round-trip delays cancel by construction.

The model is trained on the same pre-emphasized ESR objective used for the Moog target; the deployed 128-unit model achieves -29.3 dB held-out ESR at 0 dB input and -30.3 dB at -10 dB. ESR is a sample-level waveform metric reported for comparability with prior work [6]. Representative audio examples are available at https://github.com/kencul/rtneuralCsound/tree/main/audio_examples; a formal perceptual listening study remains future work.

Digital emulation of a continuous-time nonlinearity can introduce harmonic foldback above Nyquist that degrades perceived quality even when waveform-level ESR is low. We verify the deployed model’s inherent aliasing via the Aliasing-to-Signal Ratio (ASR) metric [9]. At an 8 kHz fundamental (where the fourth harmonic at 32 kHz and above fold back into the audible band), the model measures -37.9 dB ASR, well below the threshold at which antialiased architectures [8] or teacher-student fine-tuning [10] would be warranted.

5 Real-Time Benchmarking

Benchmarks were run on an AMD Ryzen 5 5600X under an MSVC Release build at 48 kHz with 64-sample control blocks (**ksmps**). Compiled opcodes are compared against Csound’s built-in **moogladder** [1] and **rkmoog**, a custom plugin wrapping the exact $8\times$ oversampled RK4 simulation used to generate training data.

Table 1. Single-voice CPU cost, real-time polyphony, and mean static ESR at 48 kHz (**ksmps**=64). Filter neural rows (**moognn**) are trained on static cutoffs only unless labelled otherwise.

Opcode	GRU Units	CPU% / Voice	Max Polyphony	Mean Static ESR
moogladder (native DSP)	—	0.7%	141	—
rkmoog (RK4 reference)	—	2.6%	39	—
moognn32	32	5.9%	16	-39.6 dB
moognn64	64	8.6%	11	-42.3 dB
moognn128	128	16.9%	6	-45.9 dB
moognn256	256	60.0%	1	-47.9 dB
moognn128 (multi-rate LFO training)	128	16.9%	6	-49.3 dB
distnn (hardware pedal)	128	16.5%	6	-29.3 dB

Execution cost scales quadratically with hidden size due to the $\mathcal{O}(H^2)$ matrix multiplications inside the single-sample GRU loop. The deployed **moognn128** requires $\approx 24\times$ more compute per voice than **moogladder** and $6.5\times$ more than **rkmoog**, a cost paid for architectural generality: the same compiled binary (**distnn**, 16.5% CPU per voice) also models physical breadboard circuitry directly from audio recordings, which is impossible for fixed algorithmic opcodes. Sustaining 6-voice polyphony on standard consumer hardware demonstrates that recurrent neural audio models are viable beyond monophonic use.

6 Conclusion

We demonstrated real-time recurrent neural inference for VA modeling directly inside Csound scores. Compile-time graph optimization in RTNeural, combined with a multi-rate LFO training distribution for parametric filters, produces 128-unit models that emulate both an RK4 filter simulation and a hardware distortion pedal at 6-voice polyphony on consumer hardware. Two limitations bound this result. First, the polyphony ceiling is architectural rather than implementation-specific: GRU inference is $\mathcal{O}(H^2)$ per sample and strictly sequential in the hidden state, so per-voice cost cannot be amortized across a control block; the **moognn256** row of Table 1 (60% CPU per voice, single-voice only) illustrates the ceiling directly. Second, all reported accuracy figures are sample-level (ESR) or spectral (ASR) metrics used for comparability with prior work; we do not claim perceptual equivalence to the modeled devices. Audio examples are provided (Sec. 4) and a formal listening study remains future work. Future work targets a linear recurrent unit [7,8] as a lower-cost GRU replacement via diagonal state updates and no gating overhead to raise the polyphony ceiling, alongside wider training distributions and formal perceptual evaluation. Code, trained weights, and evaluation data are available at <https://github.com/kencul/rtneuralCsound>.

References

1. Huovilainen, A.: Non-linear digital implementation of the Moog ladder filter. In: Proceedings of the 7th International Conference on Digital Audio Effects (DAFx-04), pp. 61–64. Naples, Italy (2004)
2. Chowdhury, J.: RTNeural: Fast Neural Inferencing for Real-Time Systems. arXiv preprint arXiv:2106.03037 (2021)
3. Peussa, A., Damskäg, E.P., Sherson, T., Mimilakis, S.I., Juvela, J., Gotsopoulos, A., Välimäki, V.: Exposure bias and state matching in recurrent neural network virtual analog models. In: Proceedings of the 24th International Conference on Digital Audio Effects (DAFx20), originally scheduled for 2020 but postponed to 2021 due to the COVID-19 pandemic, pp. 284–291. Vienna, Austria (2021)
4. Damskäg, E.P., Juvela, L., Välimäki, V.: Real-time modeling of audio distortion circuits with deep learning. In: Proceedings of the 16th Sound & Music Computing Conference (SMC 2019), pp. 332–339. Malaga, Spain (2019)
5. Simionato, R., Fasciani, S.: Conditioning methods for neural audio effects. In: Proceedings of the International Conference on Sound and Music Computing (SMC 2024), pp. 411–418. (2024)
6. Wright, A., Damskäg, E.P., Välimäki, V.: Real-time black-box modelling with recurrent neural networks. In: Proceedings of the 22nd International Conference on Digital Audio Effects (DAFx-19), pp. 1–8. Birmingham, UK (2019)
7. Orvieto, A., Smith, S.L., Gu, A., Fernando, A., Gulcehre, C., Pascanu, R., De, S.: Resurrecting recurrent neural networks for long sequences. In: Proceedings of the 40th International Conference on Machine Learning (ICML), pp. 26580–26608. Honolulu, Hawaii (2023)
8. Esqueda, F., Murai, Y.: Antialiased black-box modeling of audio distortion circuits using real linear recurrent units. In: Proceedings of the 28th International Conference on Digital Audio Effects (DAFx25). Ancona, Italy (2025)
9. Sato, R., Smith, J.O.: Aliasing reduction in neural amp modeling by smoothing activations. In: Proceedings of the 28th International Conference on Digital Audio Effects (DAFx25). Ancona, Italy (2025)
10. Carson, A., Wright, A., Bilbao, S.: Anti-aliasing of neural distortion effects via model fine tuning. In: Proceedings of the 28th International Conference on Digital Audio Effects (DAFx25). Ancona, Italy (2025)
11. Fasciani, S.: A universal tool for generating datasets from sound synthesizers. In: Proceedings of the 22nd Sound and Music Computing Conference (SMC 2025), pp. 303–310. Graz, Austria (2025)
12. Diakopoulos, D.: MoogLadders: a collection of digital Moog ladder filter models. GitHub repository (2015). <https://github.com/ddiakopoulos/MoogLadders>. The RK4 implementation used in this work (RKSimulationMoog) is due to M. Puckette.